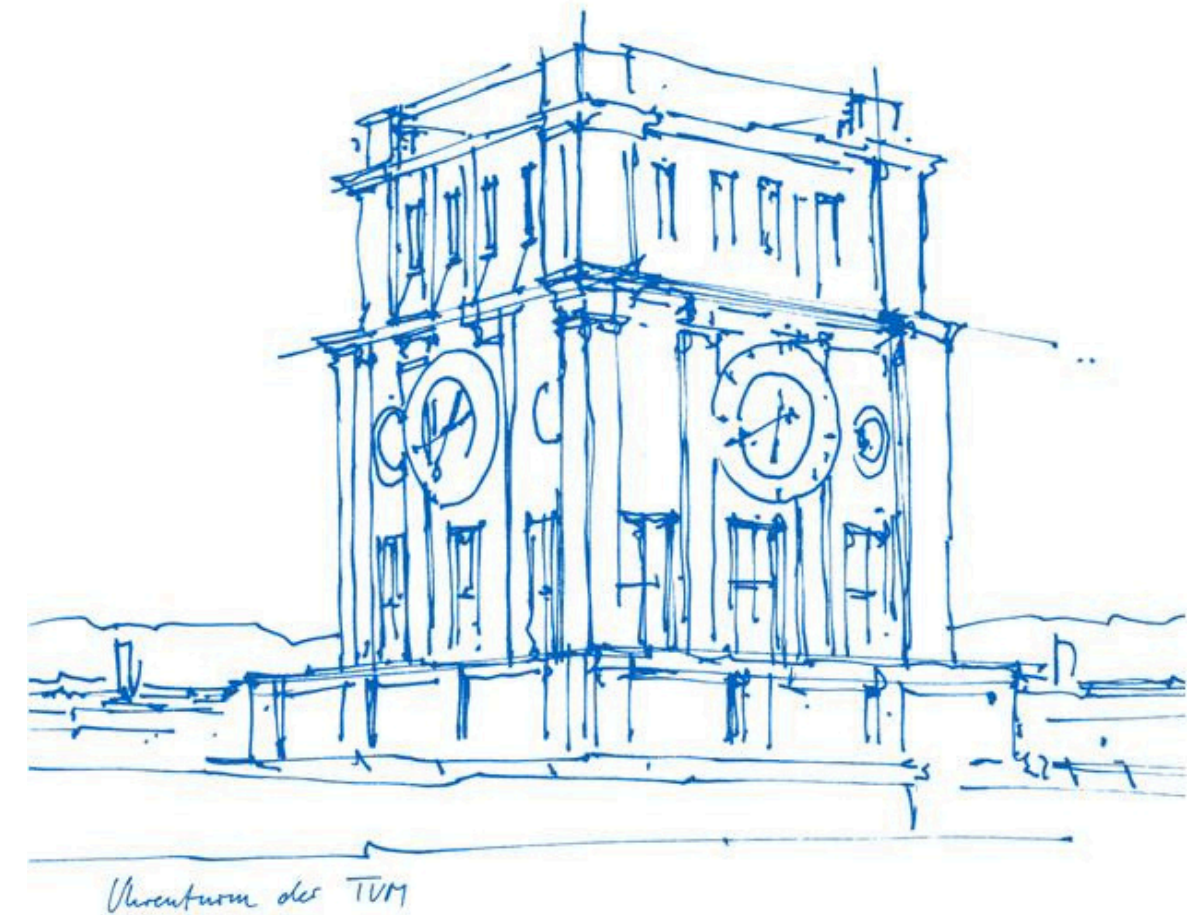


RISC-V Softcore on PYNQ-Z2

Jing Cheng

Linzhi Li

Yingtong Tian



Project Objective

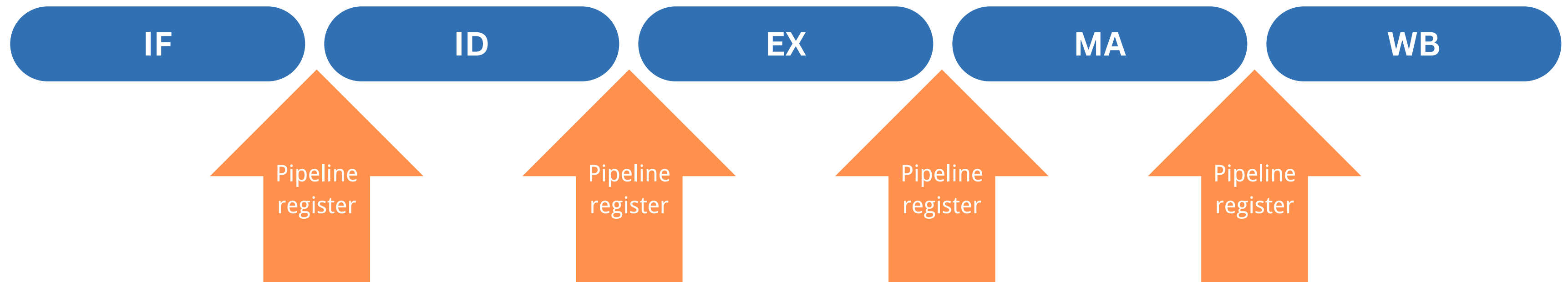
To implement and verify a pipelined RISC-V softcore on the PYNQ-Z2 FPGA board using a hardware-in-the-loop verification system.

The ARM Processing System acts as the host and testbench, while the custom RISC-V processor runs in the Programmable Logic.

Design Requirements

- Five-stage pipelined **RISC-V core**: IF, ID, EX, MA and WB
- Partial support of the RV32I instruction set
- Separate **instruction/data BRAMs**
- **AXI SmartConnect** and **AXI BRAM Controller** for PS–PL memory access
- **AXI GPIO** for active-low reset and CPU start control
- DONE magic number written by the RISC-V program to a status address
- Jupyter Notebook for program loading, data injection, polling and verification

Five-Stage Pipeline Architecture



Harzard Handling

Data hazards

- Forwarding unit from MEM/WB to EX

Load-Use hazard

- Hazard unit detects dependency
- PC freeze
- IF/ID stall
- ID/EX flush

Key challenge

- Adapting the Pipeline to Synchronous FPGA BRAM

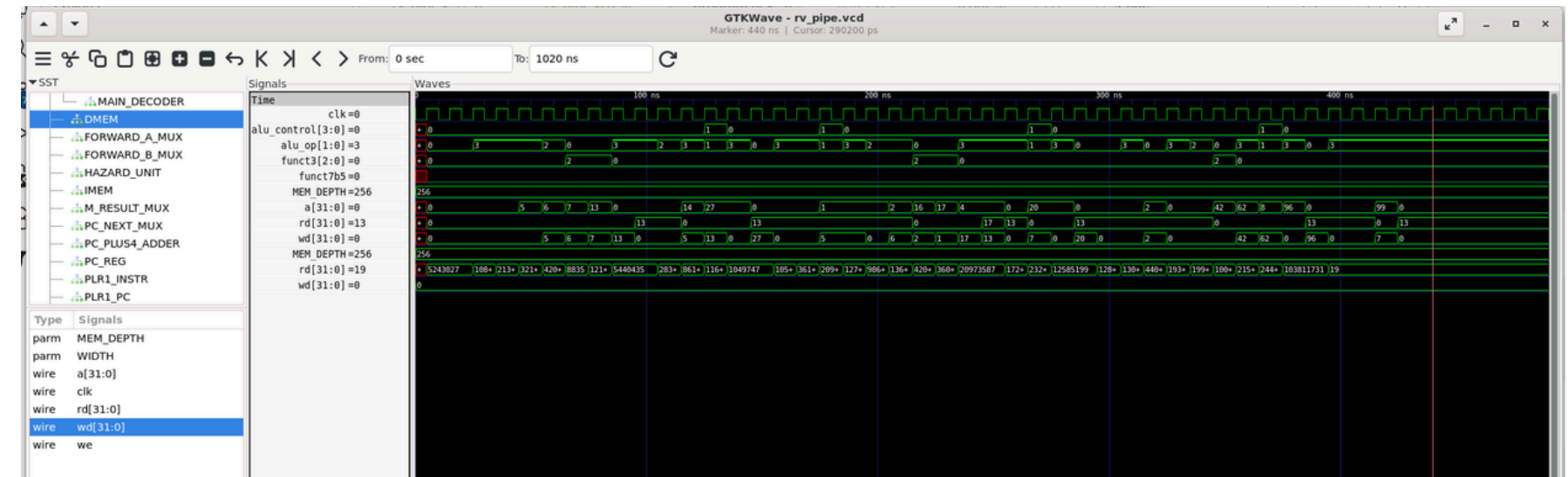


Figure: Waveform of the RISC-V Pipeline Hazard Test

System Architecture

- ZYNQ Processing System (PS)
- Processor System Reset
- AXI SmartConnect
- AXI BRAM Controllers*2
- AXI GPIO (reset control)
- Block Memory Generators*2
- Custom RISC-V IP (PL)

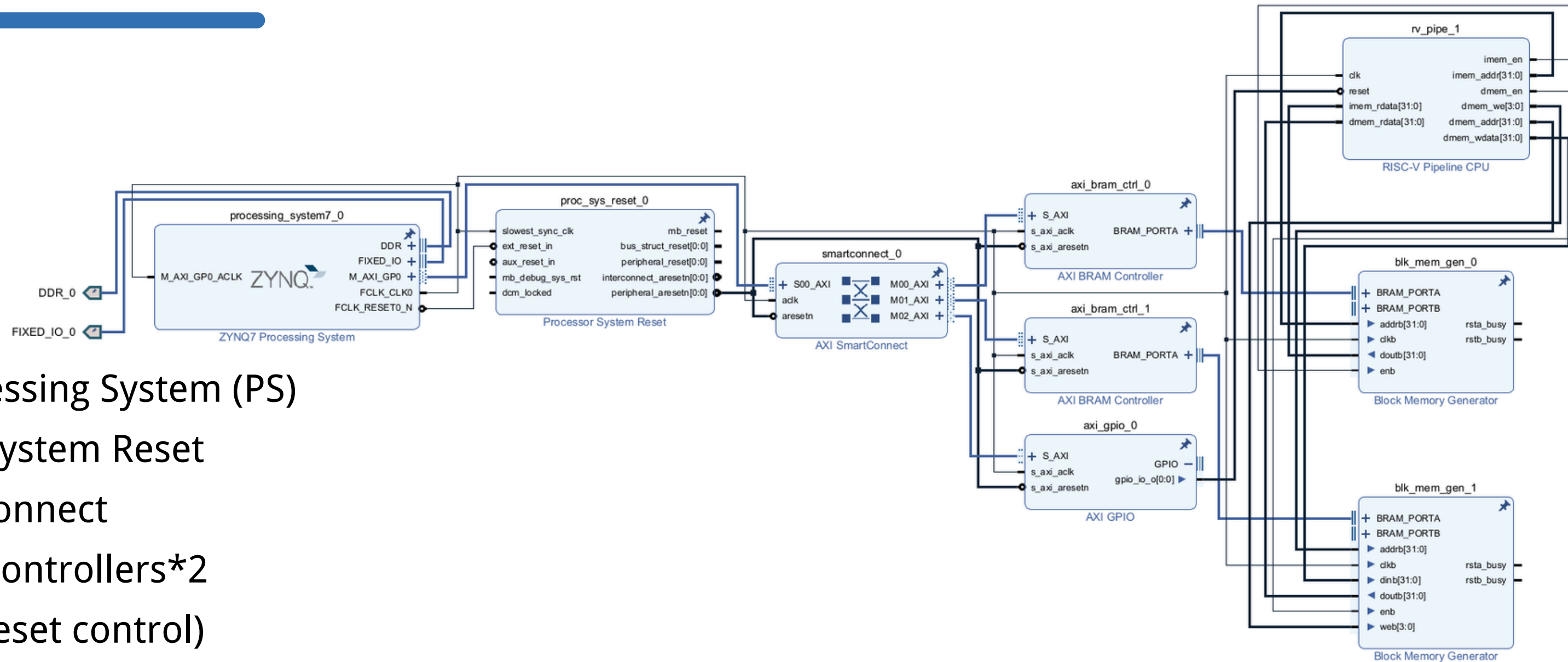


Figure: PS-PL Block Design

FPGA Implementation Results

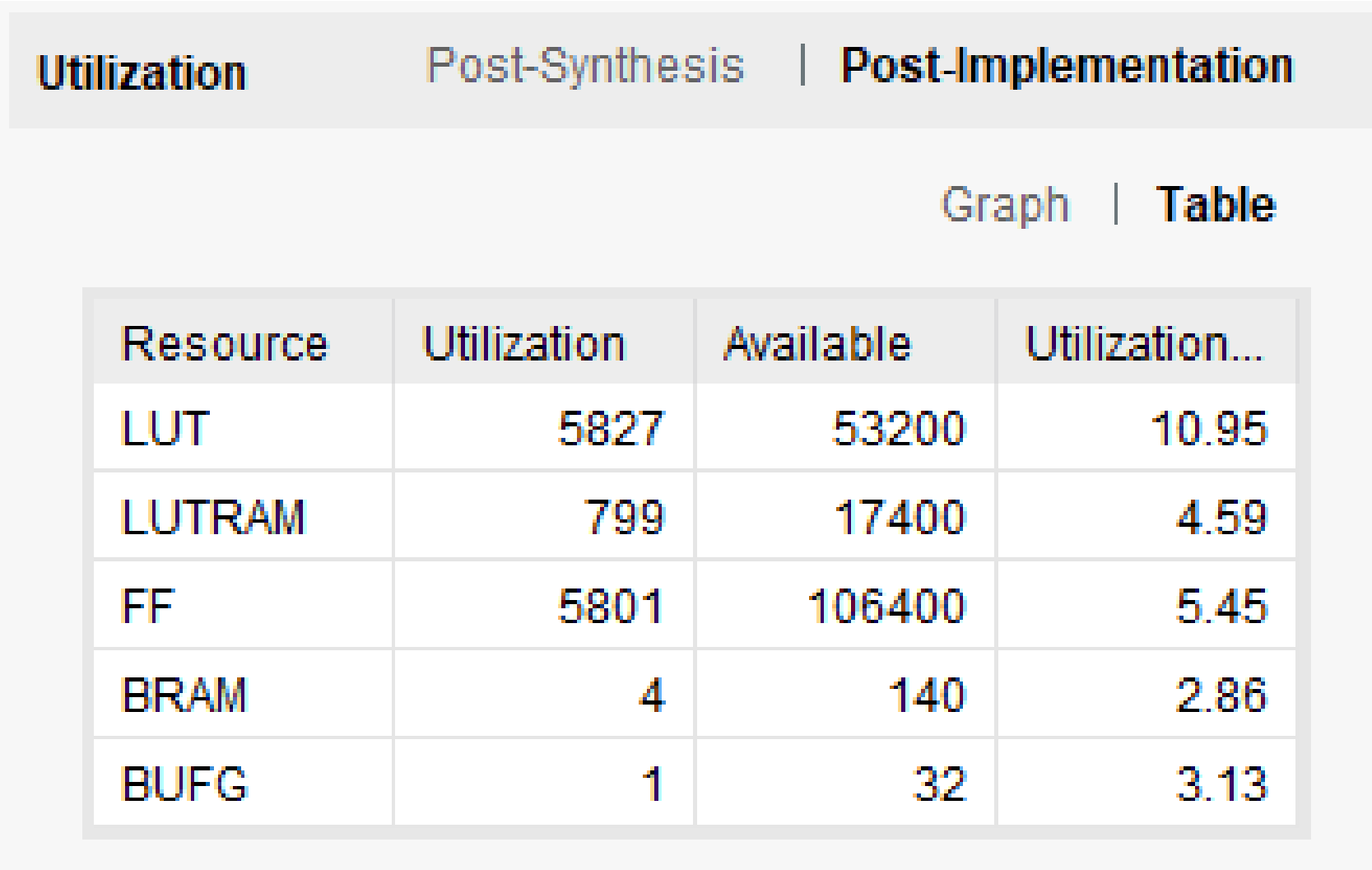


Figure: Implementation Results

Hierarchy	LUTs	Interpretation
PLR2_RS1	290	Operand preparation and optimized control logic
PLR2_EXT	126	Immediate generation and related logic
PLR2_ALU_CONTROL	124	Decode and ALU-control logic
PLR1_INSTR	108	IF/ID instruction-path logic
REGFILE	44 LUTRAM	Distributed register storage

Figure: Hierarchical Utilization

Timing Performance

Metric	Result
Worst Negative Slack (WNS)	+3.886 ns
Worst Hold Slack (WHS)	+0.024 ns
Total Negative Slack (TNS)	0 ns
Timing Violations	0
Timing Status	All timing constraints met
Target Clock Period	20.000 ns
Target Frequency	50 MHZ
Estimated Max Frequency	62.1 MHZ

Figure: Timing Results

Critical Path Analysis

Project Summary	
Device	Path 1 - timing_1
Summary	
Name	Path 1
Slack	3.886ns
Source	design_1_i/blk_mem_gen_1/U0/inst_blk_mem_gen/gnbram.gnative_mem_map_bmg.native_
Destination	design_1_i/rv_pipe_1/inst/PLR2_PC_P4/q_reg[24]/R (rising edge-triggered cell FDRE cloc
Path Group	clk_fpga_0
Path Type	Setup (Max at Slow Process Corner)
Requirement	20.000ns (clk_fpga_0 rise@20.000ns - clk_fpga_0 rise@0.000ns)
Data P...Delay	15.201ns (logic 4.158ns (27.354%) route 11.043ns (72.646%))
Logic Levels	10 (LUT3=1 LUT4=1 LUT5=4 LUT6=4)
Clock ... Skew	-0.182ns
Clock U...tainty	0.302ns

- Source: BRAM output
- Destination: ID/EX pipeline-register control input
- Logic levels: 10

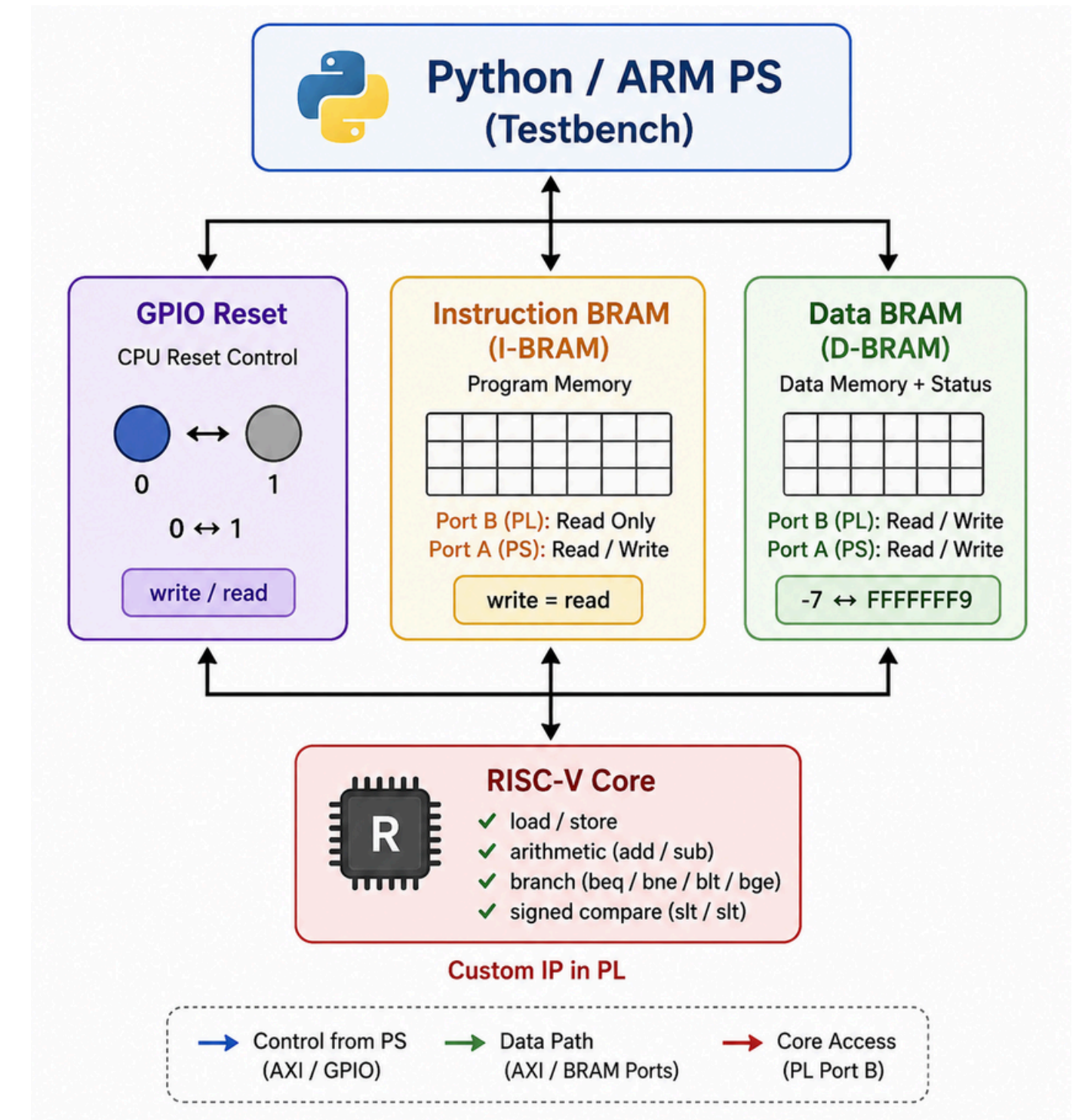
- Total path delay: 15.201 ns
- Logic delay: 4.158 ns (27.4%)
- Routing delay: 11.043 ns (72.6%)
- WNS: +3.886 ns

- Dominate Bottleneck: Routing Delay

Figure: Timing Path Summary

Hardware-in-the-Loop Verification

1. Load the bitstream and hold the RISC-V core in reset
2. Write machine code into Instruction BRAM
3. Write 32 signed integers into Data BRAM
4. Release reset and poll the completion flag
5. Read the sorted array and compare it with Python sorted()



RISC-V instructions

Executive Program Overview

- Performs an **in-place Bubble Sort** on 32 elements stored at base address `0x1000`.
- Utilizes a nested loop structure: 31 outer passes (`i`) and `x3` inner comparisons (`j`).
`x5`
- Completes by writing status flag to RA `0xCAFEBAFE` enters a self-loop. `0x2000`

Register Mapping

Register	Role / Variable	Value / Initial State
<code>x2 / x3</code>	Outer Pass Limit / Counter (<code>i</code>)	<code>x2 = 31</code> , <code>x3 = 0 .. 31</code>
<code>x4 / x5</code>	Inner Limit (<code>31-i</code>) inner (<code>j</code>)	<code>x4 = x2 - x3</code> , <code>x5 = 0 .. x4</code>
<code>x6</code>	Array Pointer Address	Base <code>0x1000</code>
<code>x7 / x8</code>	Loaded Array Values	<code>array[j]</code> , <code>array[j+1]</code>

Pipeline Hazard Handling

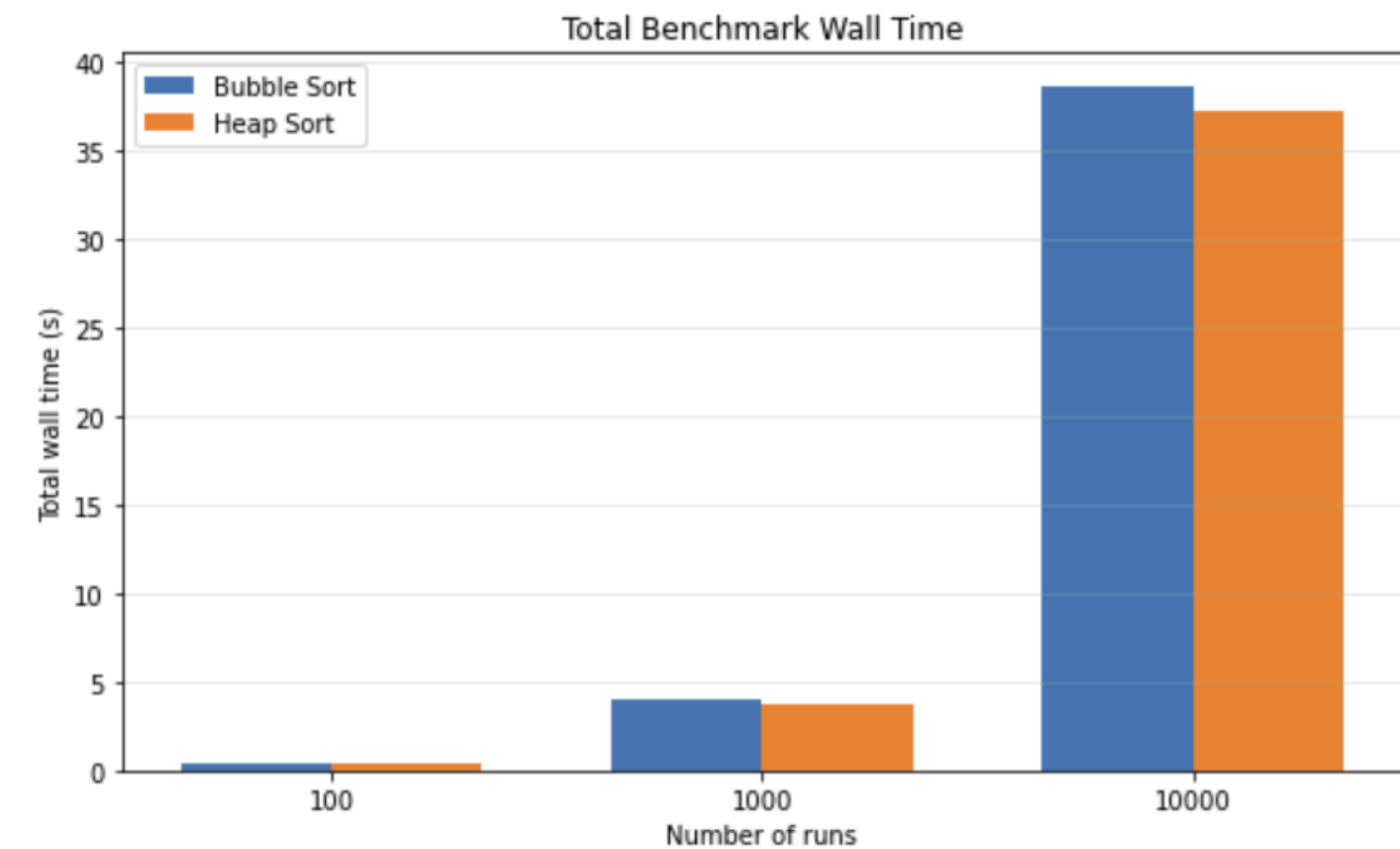
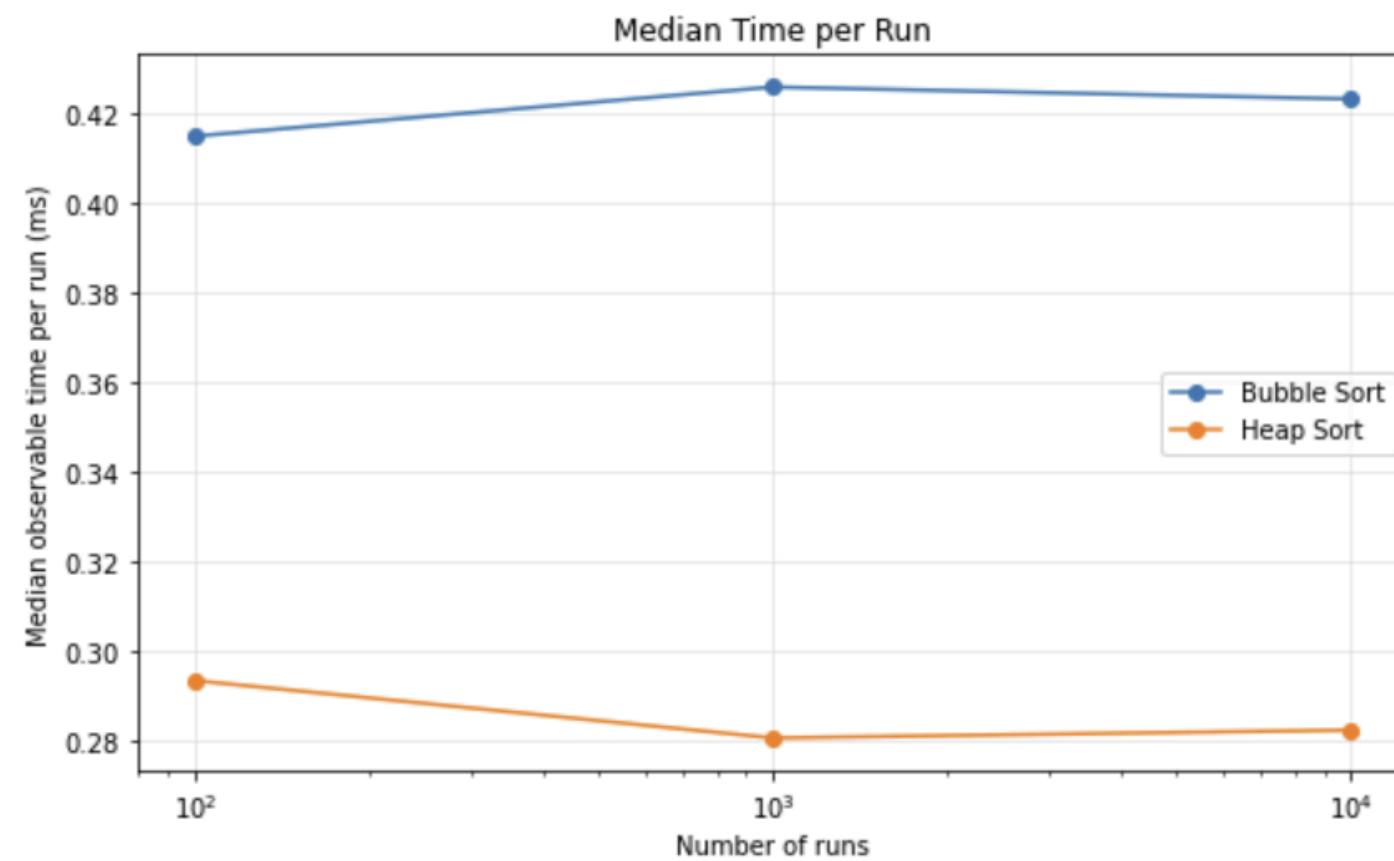
- Inserts explicit (0: `nop` 0013) delays to accommodate memory load latency (`lw`).
- Prevents pipeline stalls during register write-back before conditional branching (`slt`, `beq`).

RISC-V Program Construction & Execution Flow

- 1 Initialize Counters & Base Pointer `addi x2=31, x3=0, lui x6=0x1000`
↓
- 2 Calculate Inner Bounds & Reset Counter `sub x4, x2, x3 (31-i), addi x5, x0, 0`
↓
- 3 Load Memory & Pipeline NOP Delay `lw x7, 0(x6), lw x8, 4(x6) + 2x nop`
↓
- 4 Compare & Swap Elements (if `x8 < x7`) `slt x9, x8, x7 -> sw x8, 0(x6), sw x7, 4(x6)`
↓
- 5 Advance Array Pointer & Inner Loop `addi x6, x6, 4; addi x5, x5, 1; check j==x4`
↓
- 6 Increment Outer Pass Counter `addi x3, x3, 1; repeat until i==31`
↓
- 7  Write Completion Flag & Halt CPU `[0x2000] = 0xCAFEBAFE -> jal x0, 0 (Infinite Loop)`

 Software hazards handled via NOPs for CPU pipeline execution.

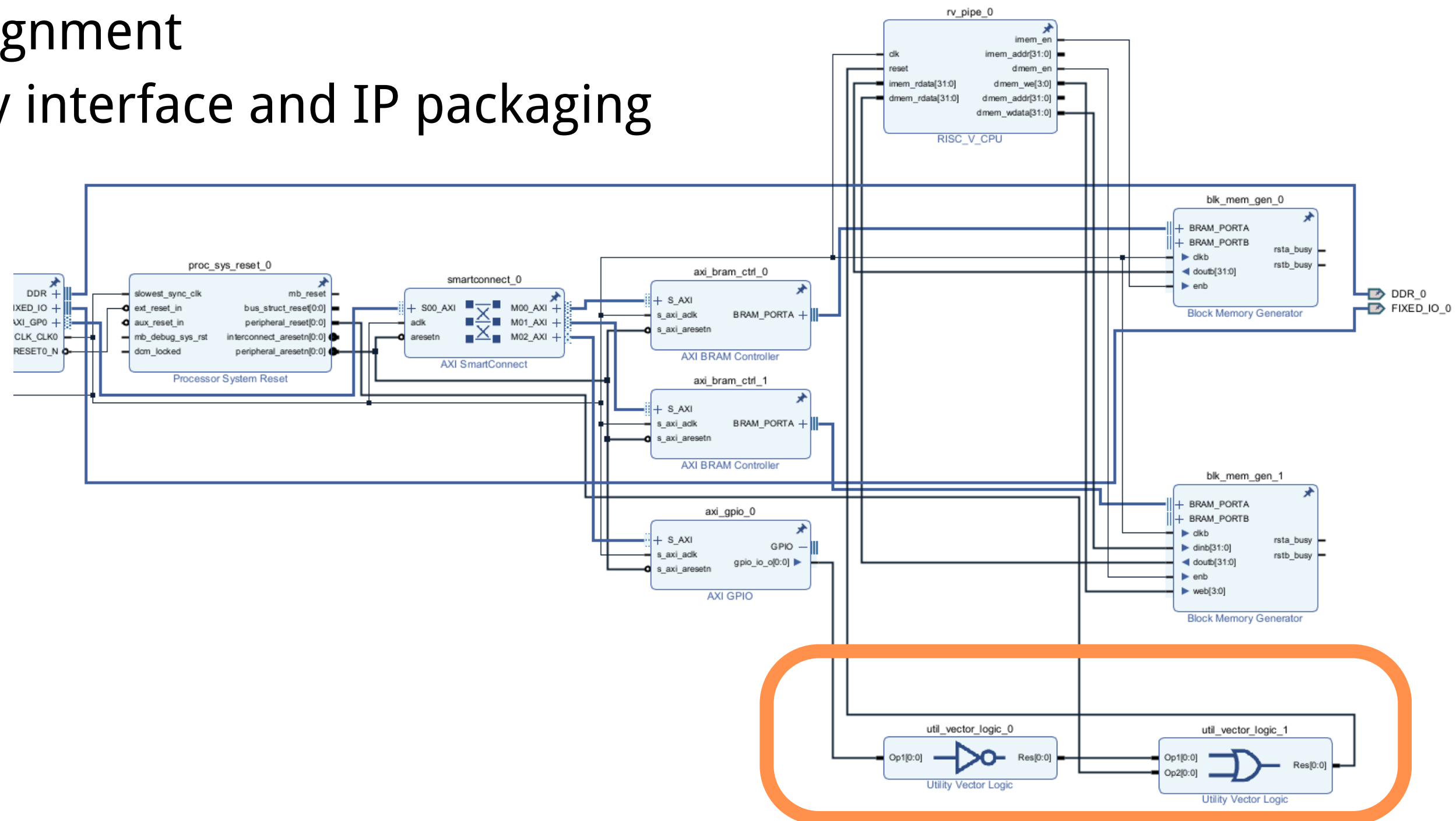
Performance between two Sort Algorithm



Challenges

Implementation Challenges:

- Reset Polarity Alignment
- External memory interface and IP packaging



Further Discussion

1. Branch performance
2. Memory timing
3. Performance counters could be added
4. Complete RV32I implementation

Conclusion

Achievements

- Functional 5-stage pipelined RISC-V on PYNQ-Z2
- Successful PS-PL hardware integration
- Fully automated HIL verification

Key Lessons

- FPGA hardware differs from simulation
- Consistent reset logic simplifies system integration
- System-level debugging is important